ApplyMate: A Unified AI-Driven Platform for Automated Scholarship and Job Applications

FYP Code: 23F-13

Software Design Specification

BY

HAMADULLAH SAMO, CMS. ID: ***023-23-0041***

MARYAM KHATOON, CMS. ID: ***053-23-0021***

HASSAN ALI, CMS. ID: ***023-23-0046***



Supervised by: Dr. Saif Hassan _____

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY

SUKKUR IBA UNIVERSITY

NOVEMBER 2026

Document History

Table of Contents

Contents

1. INTRODUCTION 1

1.1. PURPOSE OF DOCUMENT 1

2. OVERALL SYSTEM DESCRIPTION 3

2.1. PROJECT BACKGROUND 3

2.2. PROJECT OBJECTIVES 3

3. SYSTEM ARCHITECTURE 5

4. DOMAIN MODEL 8

5. CLASS DIAGRAM 10

6. DATABASE DIAGRAM 14

7. ENTITY RELATIONSHIP DIAGRAM (ERD) 16

8. SEQUENCE DIAGRAM 17

8.1. SIGN UP AND LOG IN 17

8.2. APPLY WITH THE AI AGENT 18

8.3. IMPORT PROFILE FROM CV 19

8.4. ASK APPLYMATE CHATBOT 20

9. SYSTEM INTERFACE DESIGN 20

9.1. LANDING PAGE 21

9.2. LOGIN AND SIGN-UP 21

9.3. DASHBOARD 22

9.4. PROFILE AND DOCUMENTS 23

9.5. SCHOLARSHIPS 24

9.6. JOBS 25

9.7. OPPORTUNITY DETAILS 25

9.8. PERMISSION DIALOG 26

9.9. AGENT RUN: READY TO SUBMIT 27

9.10. AGENT RUN: YOUR TURN 28

9.11. APPLICATION TRACKER 29

9.12. AGENT ACTIVITY 29

9.13. NOTIFICATIONS 30

9.14. ASK APPLYMATE CHATBOT 31

10. TEST CASES 33

1. Introduction

1.1. Purpose of Document

This document explains how we have designed ApplyMate, our final year project (23F-13). ApplyMate is a web platform for Pakistani students. A student fills in one profile, sees the scholarships and jobs that suit them, and an AI agent running on their own computer fills the application forms on the real portals.

In the proposal we described what the system should do. Here we describe how it does it: the overall architecture, the main concepts and classes, the database, the important interactions between components, the screens, and the test cases we will use to check the system.

The main readers are our supervisor and the evaluation committee. We will also use it ourselves as a reference while building and testing. Everything described here matches the current working prototype (version 1.3.0).

Definitions, acronyms, and abbreviations

Term	Definition
ApplyMate	The name of this FYP system: a unified platform for automated scholarship and job applications.
Agent	The software component that opens an application website in a browser, reads the form and fills it from the student's profile.
Angular Material	A set of web form components (for example dropdowns with a search box) used by the HEC Scholarship Portal. The agent knows how to operate them.
API	Application Programming Interface. Here, the REST endpoints under /api/* exposed by the Express server.
CAPTCHA	A test that websites use to tell humans and bots apart. The agent never solves it; the student does.
CEFR	Common European Framework of Reference for languages. Language levels from A1 to C2, used in the profile and on European forms.
CNIC	Computerised National Identity Card (Pakistan).
CRUD	Create, Read, Update, Delete.
CV	Curriculum Vitae (résumé).
DAAD	German Academic Exchange Service. Its scholarships are applied for through the DAAD Portal.
Domicile	The province or district a Pakistani citizen officially belongs to (for example "Sindh Urban"), asked on HEC and government forms.
EACEA	European Education and Culture Executive Agency, which publishes the Erasmus Mundus catalogue as an RSS feed.

Term	Definition
ERD	Entity Relationship Diagram.
HEC	Higher Education Commission of Pakistan. Pakistani students apply for Stipendium Hungaricum first on the HEC Scholarship Portal.
Honeypot field	A hidden form field meant to trap bots. The agent detects and skips it.
JWT	JSON Web Token. A signed token the browser sends with every request to prove who the user is.
LLM	Large Language Model. ApplyMate uses qwen3:4b, run locally through Ollama.
MERN	MongoDB, Express, React, Node.js: the web stack used by ApplyMate.
MongoDB	A document (NoSQL) database. Data is stored in collections of JSON-like documents.
OCR	Optical Character Recognition. Tesseract is used to read text from scanned CVs.
Ollama	A tool that runs LLMs on the local machine and serves them through an HTTP API.
OTP	One-Time Password, e.g. an email or SMS verification code.
Playwright	A browser-automation library. The agent uses it to read and fill web forms.
Profile key	A named piece of profile data, e.g. personal.cnic. The LLM sees key names; the program fills in the actual values.
RAG	Retrieval-Augmented Generation. The chatbot first retrieves matching listings, then the LLM answers from those only.
Recorded route	The clicks to a programme's application form, recorded once by our team with "npm run teach" and replayed by the agent for every student.
Reset code	A 6-digit code emailed to a student who forgot the password. It works for 15 minutes; only its hash is stored.
Route	The saved address of a scholarship's application form, learned by the agent and reused for all students.
RSS	Really Simple Syndication, an XML feed format.
Saved answer	A student's answer to a question a form asked. It is stored in the profile and reused on every form that asks the same question.
SAP UI5	A web framework used by the DAAD Portal. The agent reads and fills its form components.
SDS	Software Design Specification (this document).
SMTP	Simple Mail Transfer Protocol, used to send email notifications through Gmail.
SPA	Single-Page Application.
Tracker	The Kanban board where a student follows each opportunity from "Saved" to "Decision".
UI	User Interface.

2. Overall System Description

2.1. Project Background

Many Pakistani students apply every year for scholarships such as Fulbright, Chevening, Stipendium Hungaricum, DAAD and Erasmus Mundus, and for jobs on sites like Rozee.pk. Every portal asks for an account and has its own form, its own field names and its own rules for documents. If a student applies to five places, they end up typing the same name, CNIC, phone number, education and work history five times.

We saw three problems with this. It takes a long time, especially for students in smaller cities who depend on mobile data or a shared computer. It also leads to mistakes such as a date in the wrong format, a skipped field or a missing document, and these small mistakes are a common reason applications get rejected or left unfinished. Finally, the information students need is spread across many official pages, so deadlines are missed and students sometimes apply for programmes they are not eligible for.

The tools available today only cover part of this. Scholarship listing sites tell you what exists but stop there. Browser autofill works for simple fields with standard names and fails on most real application forms. Newer AI web agents can do more, but they usually rely on paid cloud models, which means sending personal data like CNIC numbers to someone else's servers. We could not find anything built for Pakistani applicants that brings together one profile, matching and safe form filling while keeping the data on the student's own machine.

That gap is where ApplyMate comes in. Research on LLM web agents such as Mind2Web, WebArena and SeeAct has shown that language models can understand what a form field is asking for, and small models can now run offline on an ordinary laptop. Putting these two things together makes a private and low-cost form-filling assistant possible.

2.2. Project Objectives

Our objectives for ApplyMate are:

- Let the student enter their personal, address, education, work and preference details only once, and upload their CV, transcript, CNIC copy and photo once. A CV can also be imported to fill most of the profile automatically.
- Bring scholarships and jobs into one place, with deadlines, status, eligibility, required documents and application steps. The scholarships are Fulbright, Chevening, Stipendium Hungaricum (applied for through the HEC Scholarship Portal), seven DAAD programmes for Pakistani applicants (applied for through the DAAD Portal) and Erasmus Mundus joint Master's programmes. For jobs, the current version has a practice job on our demo portal, and more job listings will be added later.
- Give each opportunity a match score from 0 to 100 and show the reasons behind it, for example "Masters level" or "field: computer science", so the student can see why it was suggested. Programmes the student cannot apply for (wrong level or subject, not enough

work experience, only for academics, or only for former scholarship holders) are marked as not suitable, with the reason.

- Build an agent that, after the student agrees, finds the application form, works out what belongs in each field using an offline LLM, and fills text boxes, dropdowns, radio buttons, checkboxes, date pickers and file uploads. On portals such as HEC it also adds the student's degrees one by one to the education history.
- Keep the student in charge. The agent stops for sign-in, verification codes, CAPTCHAs and document uploads. Questions that only the student can answer are asked inside ApplyMate, and the answers are saved so the student never has to answer the same question twice. The agent never ticks declarations and never presses the final Submit button.
- Protect privacy. The LLM runs locally. It is shown field labels, profile key names and a few general facts (such as nationality, degrees and language level), but never the CNIC, passport number, phone, email, date of birth or street address.
- Help students keep track of their applications through a tracker board, in-app notifications and optional email reminders.
- Let students manage their own account: change their name and password in Account settings, and reset a forgotten password with a 6-digit code sent to their email.
- Answer students' questions about the listed opportunities through a chatbot called "Ask ApplyMate".

By the end of the project we want a working web application where a student can go from finding a scholarship to having its form filled and ready to submit, without leaving ApplyMate and without typing their details again.

3. System Architecture

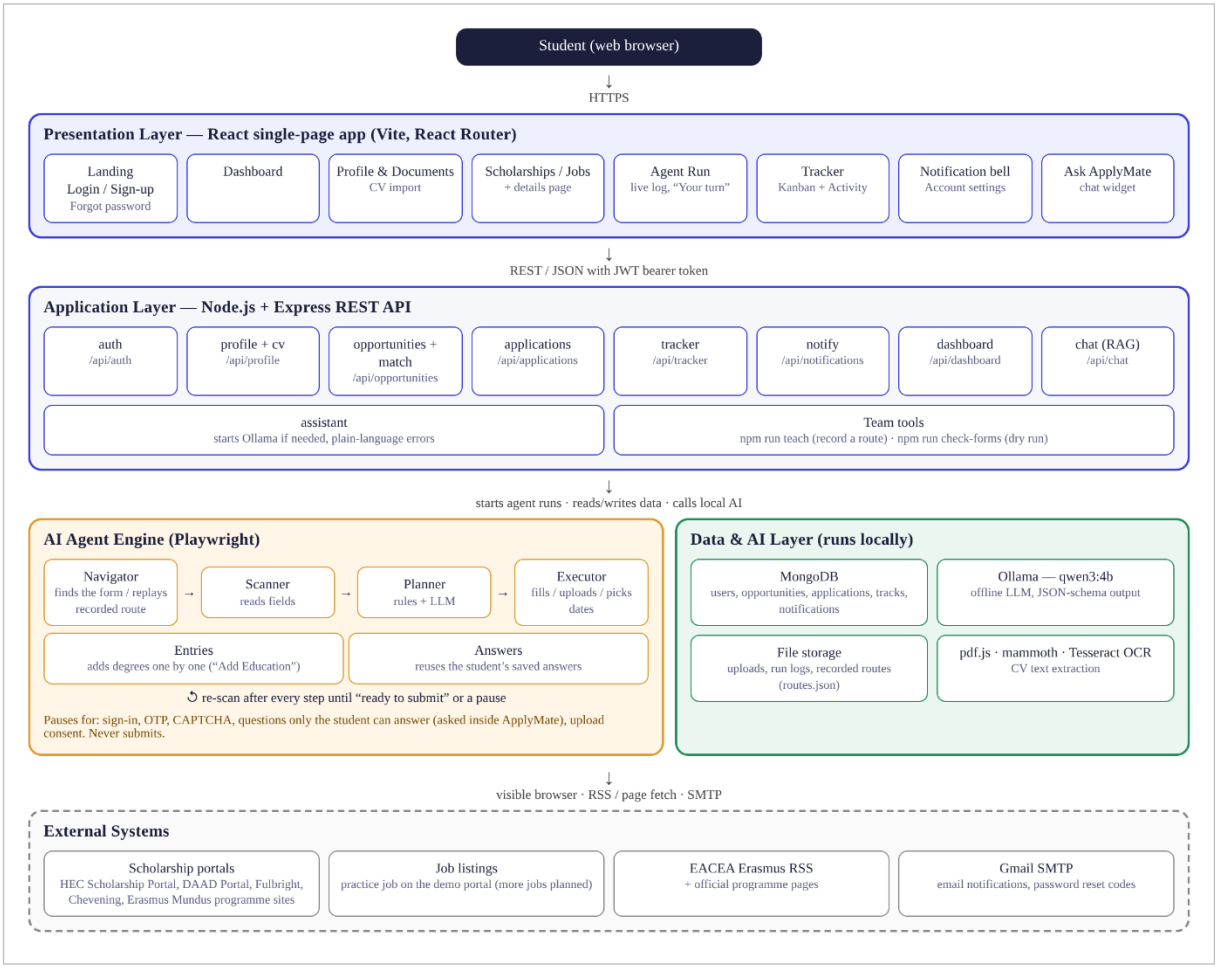


Figure 3.1: Layered system architecture of ApplyMate

ApplyMate uses a layered client-server architecture, with the form-filling agent kept as a separate engine. Apart from the external portals, everything runs on the student's own computer. This keeps personal data local and lets the AI work without an internet connection.

Presentation layer

The front end is a React single-page application built with Vite. Visitors see the landing, login, sign-up and forgot-password pages. Once logged in, the student works inside /app, which has the Dashboard, Scholarships, Jobs, Opportunity Details, Agent Run, Application Tracker, Agent Activity, Profile and Account settings pages. The notification bell and the chat widget are available on every page. The front end only talks to the server through JSON requests, and each request carries the login token.

Application layer

The back end is a Node.js server using Express. It is split into eight route modules: auth, profile, opportunities, applications, tracker, notifications, dashboard and chat. The auth module also handles account settings (name and password) and resetting a forgotten password with a code sent by email. Apart from sign-up, login and password reset, every request goes through a middleware called requireAuth, which checks the token and loads the user. A helper module called assistant makes sure Ollama is running before the agent starts (it starts Ollama if needed) and turns technical errors into plain messages for the student. Another helper, questions, keeps the list of questions each programme's form has asked clean, without broken labels or duplicates. The same server also serves the built React app and a few local demo forms that we use for testing. Two command-line tools are used by our team: "npm run teach" records the route to a programme's form once, and "npm run check-forms" checks, without filling anything, whether the agent can reach the form of every open scholarship.

AI agent engine

The agent is started by the applications module and works in a loop. If our team has recorded the route to a programme's form, the Navigator replays it step by step (for example, on the HEC portal: dashboard, then "Hungarian Scholarship Program"). Otherwise it starts from the programme's website and follows links like "Admissions" and "Apply online" until it reaches the form, or goes straight there if a route was saved by an earlier run. The Scanner then reads every field on the page, including fields inside frames, pop-ups and new tabs, and components such as Angular Material dropdowns and SAP UI5 forms. The Planner decides what goes in each field. It uses fixed label rules and the student's saved answers first, and asks the LLM only for the remaining fields. The LLM can choose a profile key, answer from general facts about the student, or say that the student must be asked. When a page has a list such as "Add Education", the Entries module adds each missing degree in its pop-up. The Executor carries out the actions, such as typing, selecting, picking a date, uploading or clicking Next. After every step the page is scanned again. If the agent cannot or should not continue, it pauses and waits for the student to respond on the Agent Run page.

Data and AI layer

Data is kept in MongoDB in five collections. Uploaded documents and agent run logs are saved as files, and the routes recorded by our team are kept in server/catalog/routes.json. Ollama serves the qwen3:4b model on the local machine, and every request to it includes a JSON schema so the reply can always be read by the program. The model is given field labels, profile key names and a short list of general facts about the student (nationality, gender, city and country of residence, place of birth, degrees, languages, years of experience, previous scholarships and answers the student saved). It never sees the CNIC, passport number, phone number, email, date of birth or street address. Those values are filled in by the program from the profile.

External systems

The agent fills forms on the real application portals in a browser window the student can watch, including the HEC Scholarship Portal (for Stipendium Hungaricum) and the DAAD Portal. Erasmus Mundus programmes are loaded from the EACEA RSS feed. The other scholarships come from a catalogue that our team maintains, with each cycle's opening and closing dates and a live check of their status. For jobs, the Jobs page currently shows a practice job on our demo portal. The server can still add a Rozee.pk job from its link, but this option is not shown on the Jobs page at the moment. Email notifications and password reset codes go out through Gmail.

Technology stack

Layer	Technology	Responsibility
Presentation	React 18, React Router, Vite	Single-page web app: pages, dialogs, live agent progress, chat widget.
Application	Node.js, Express, JWT, bcryptjs, multer	REST API, authentication, file uploads, business rules.
Agent	Playwright (Edge / Chromium)	Opens portals in a visible browser, reads and fills forms.
AI	Ollama + qwen3:4b (local)	Field mapping, link choice, CV fact extraction, chatbot answers. JSON-schema output.
Document processing	pdf.js, mammoth, Tesseract.js	Reads text from PDF, Word and scanned CVs.
Data	MongoDB + Mongoose	Users and profiles, opportunities, applications, tracker, notifications.
Integration	fast-xml-parser, cheerio, Nodemailer	Erasmus RSS feed, programme pages, Rozee links, Gmail SMTP.

4. Domain Model

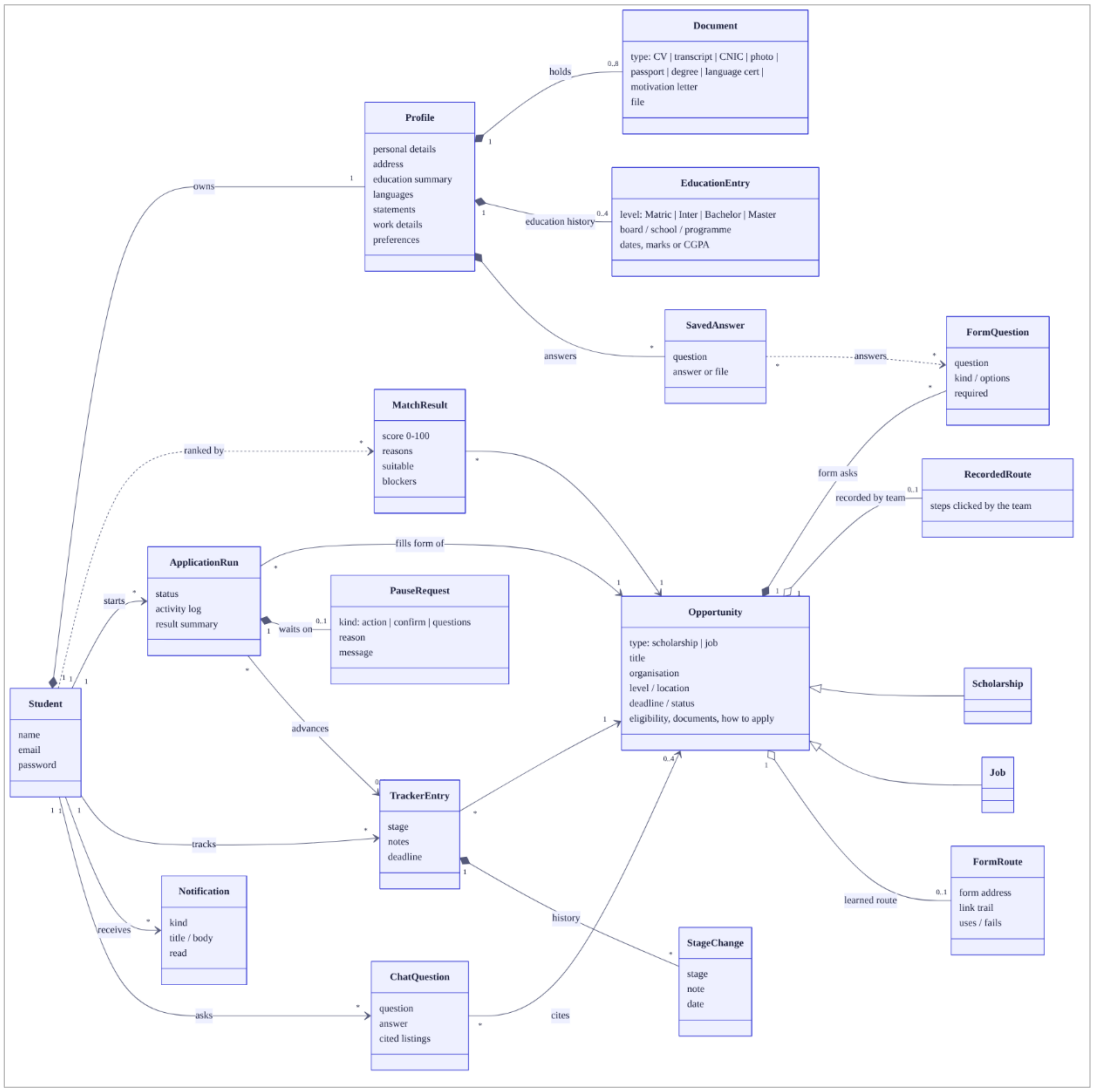


Figure 4.1: Domain model of ApplyMate

The domain model shows the main ideas in the problem and how they connect, without going into implementation. The Student is at the centre. Every student has one Profile. The profile can hold up to eight Documents, up to four EducationEntries (Matric, Intermediate, Bachelor's and Master's) and any number of SavedAnswers, which are the student's answers to questions that forms have asked. A student can start many application runs, keep many opportunities in the tracker, receive many notifications and ask the chatbot as many questions as they like.

An Opportunity is either a Scholarship or a Job. Once the agent (or a student) finds where an opportunity's application form is, that FormRoute is stored with the opportunity, so the next run, for any student, can go there directly. For some programmes our team records a RecordedRoute instead, which the agent replays step by step. An opportunity also remembers the FormQuestions

its form has asked, so other students can answer them before they apply. The MatchResult says whether the opportunity is suitable and, if not, lists the blockers. Each ApplicationRun is for one opportunity and can be waiting on at most one PauseRequest at a time. As the run progresses it moves the related TrackerEntry forward, for example to "In progress" and then "Ready to submit". Every change of stage is kept in the entry's history.

Concept	Meaning
Student	A registered user of ApplyMate.
Profile	All reusable data of the student: personal details, address, education, education history, languages, statements, work details and preferences.
Document	An uploaded file of a fixed type: CV, transcript, CNIC copy, photo, passport, degree certificate, language certificate or motivation letter.
EducationEntry	One degree of the education history (Matric, Intermediate, Bachelor's or Master's) with its board, school, dates and marks or CGPA.
SavedAnswer	The student's answer (or uploaded file) for a question a form asked, reused on every form that asks the same.
Opportunity	A scholarship or job that can be applied for. Scholarship and Job are its two kinds.
MatchResult	How well an opportunity fits a student: a score from 0 to 100, readable reasons, whether it is suitable, and the blockers if it is not. Calculated on request, not stored.
FormRoute	Where the application form of an opportunity is, learned by the agent or shown once by a student, and shared by all students.
RecordedRoute	The steps to a programme's form recorded once by our team ("npm run teach") and replayed by the agent.
FormQuestion	A question a programme's form asked that the profile could not answer. Stored without answers, so other students can prepare.
ApplicationRun	One run of the agent on one opportunity, with its status, activity log and result summary.
PauseRequest	A moment where the agent hands control to the student: an action is needed (e.g. sign-in or CAPTCHA), permission is needed (e.g. uploading documents), or the form asks questions only the student can answer.
TrackerEntry	The student's progress on one opportunity, from Saved to Decision, with notes and a personal deadline.
StageChange	One step in the history of a tracker entry.
Notification	A message to the student in the bell and optionally by email (agent needs you, form ready, deadline, etc.).
ChatQuestion	A question to Ask ApplyMate and its answer, which cites the listings it used.

5. Class Diagram

Our back end is written in JavaScript, and the code is organised into modules rather than classes. To draw the class diagrams we treat each module as a class and list its main functions as operations. A "+" marks a function the module exports and a "-" marks an internal one. The Mongoose models are shown with their main fields; the full list of fields is in the database diagram (Figure 6.1).

Server classes and data models

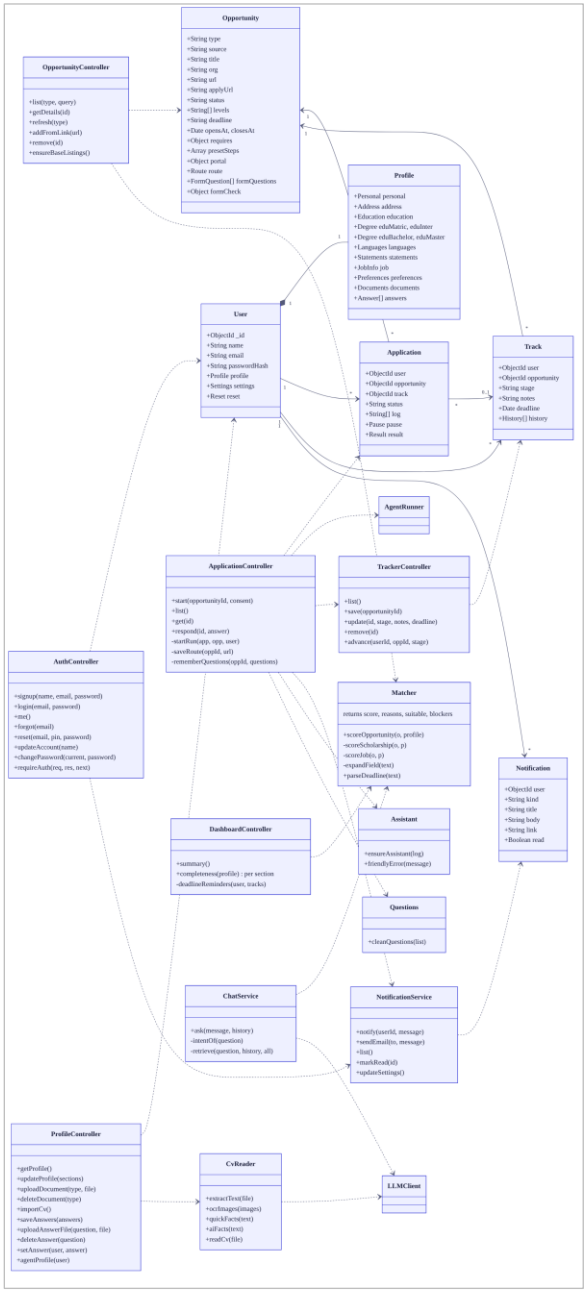


Figure 5.1: Class diagram of the server modules and data models

On the model side, the Profile is stored inside the User (composition). It also holds the education history (one degree each for Matric, Intermediate, Bachelor's and Master's), languages, statements and saved answers. Application, Track and Notification each belong to one User, and Application and Track each point to one Opportunity. An Application can also point to the Track entry that it updates. Opportunity has new fields for cycle dates, hard requirements, the recorded route, the portal's own programme names, the questions its form asked and the result of the last form check.

AuthController looks after sign-up, login and the requireAuth check, and also lets the student change their name and password and reset a forgotten password with an emailed code (sent through NotificationService). ProfileController saves the profile and documents, calls CvReader when the student imports a CV, and saves, uploads and deletes the student's answers to form questions. OpportunityController lists, refreshes and adds opportunities, using Matcher to score them and decide whether each one is suitable. Matcher also reads short degree names such as "BSCS", "MS CS", "BSIT" or "BBA" as their full subject. ApplicationController is the busiest module: it asks Assistant to make sure Ollama is running, starts the AgentRunner, passes pauses and questions to the student and their answers back to the agent, remembers which questions a programme's form asks (cleaned by Questions), saves the routes the agent learns, updates the tracker through TrackerController and sends messages through NotificationService. DashboardController prepares the home page summary, works out how complete each profile section is, and sends deadline reminders. ChatService finds the relevant listings and asks the LLM to answer.

Agent classes

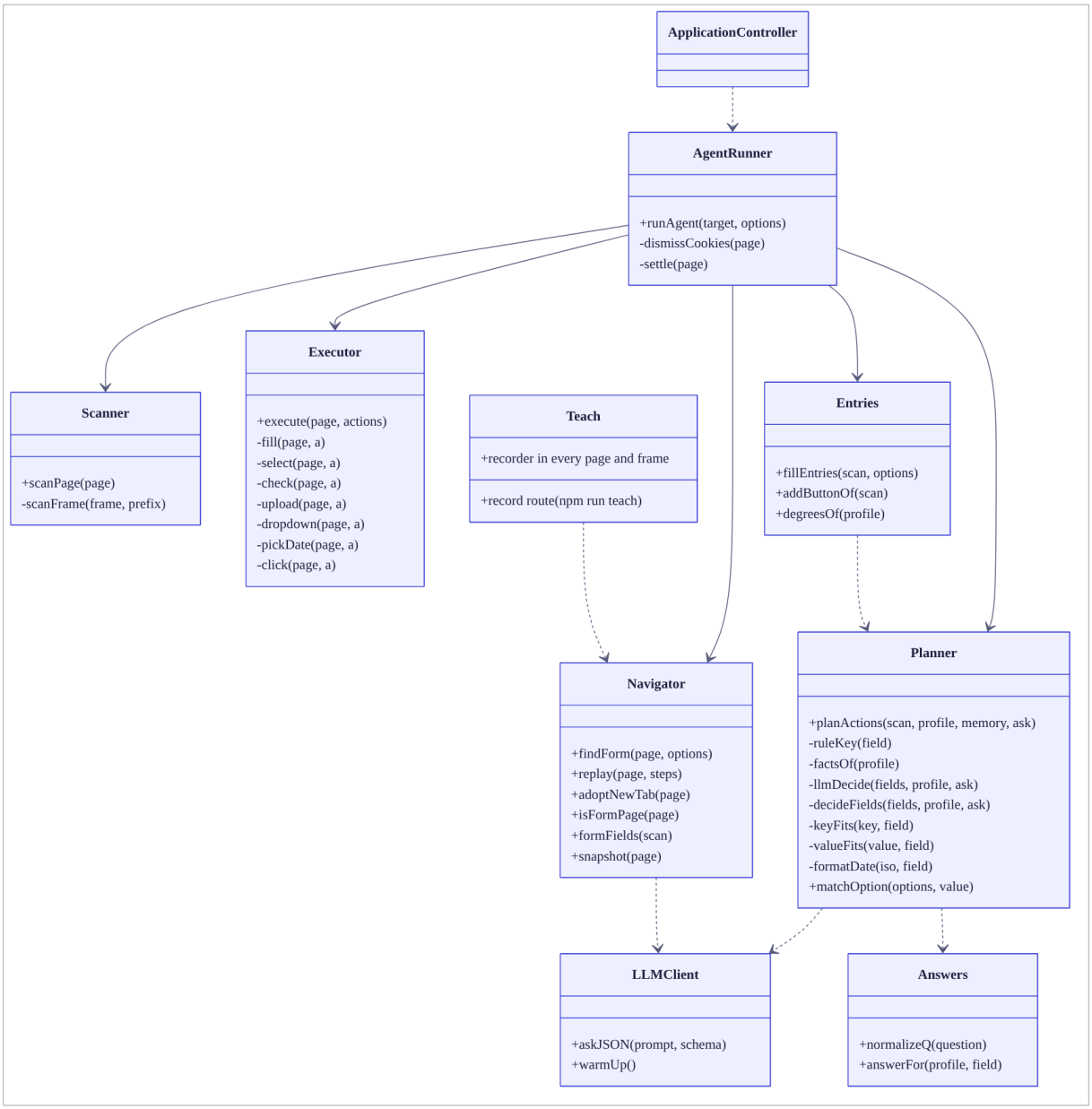


Figure 5.2: Class diagram of the AI agent engine

AgentRunner (agent/index.js) is in charge of a single run. It closes cookie banners, waits for the page to finish loading and then repeats scan, plan and execute, for at most 25 steps. Navigator looks for the form by scoring the links on a page, replays a route recorded by our team, follows new tabs, and only asks the LLM when two or more links look equally good. Scanner reads the fields on the page, including those inside frames and pop-ups. Planner decides each field using

label rules and saved answers first and the LLM second. It checks every LLM choice (a key must fit the question and a value must look right for its field), formats dates and phone numbers, checks length limits, skips honeypot fields and declarations, and decides when the agent has to stop. Entries adds the degrees of the education history one by one. Answers finds a saved answer for a question. Executor performs the actions, including calendar date pickers, and LLMClient sends every prompt to Ollama with a JSON schema and returns the parsed answer. Teach is the tool our team uses to record a route once.

6. Database Diagram

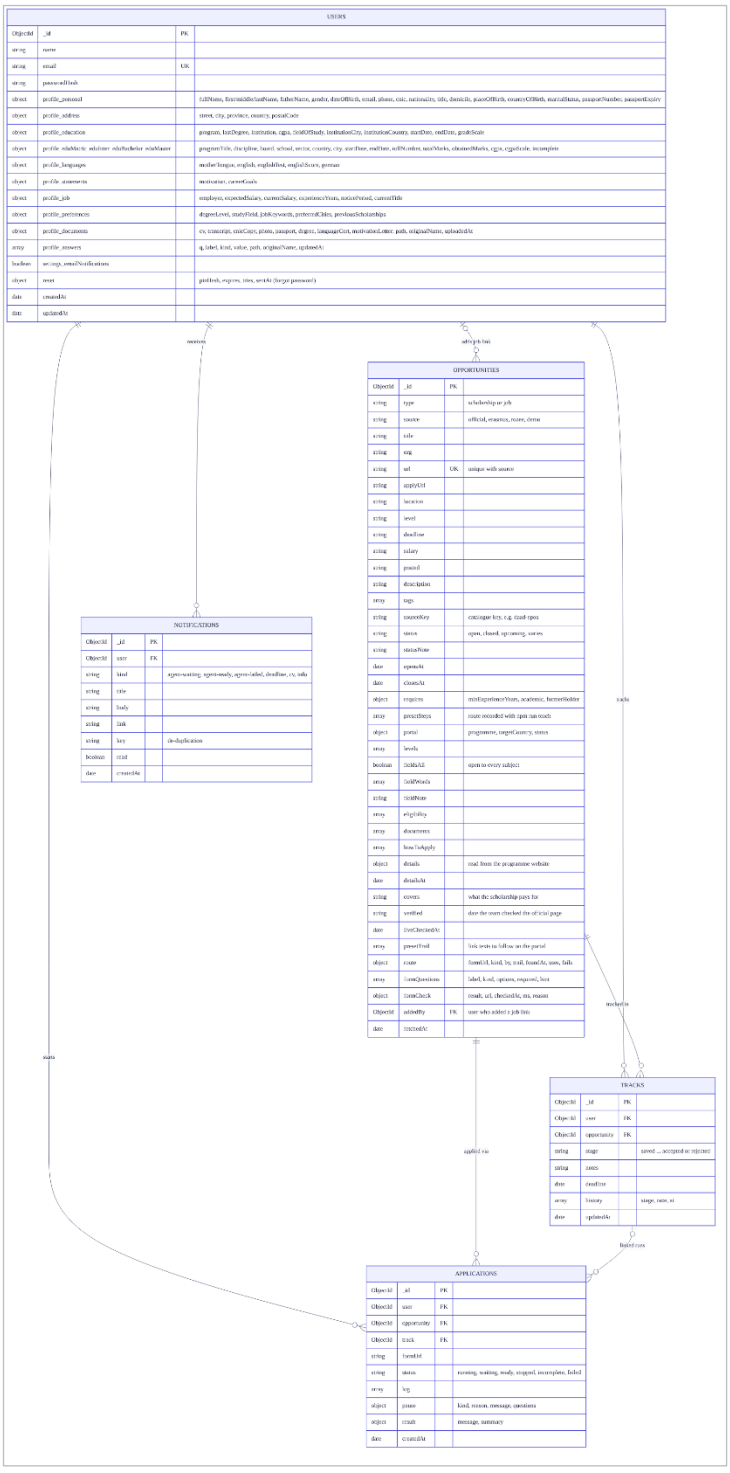


Figure 6.1: Database diagram (MongoDB collections)

We use MongoDB with Mongoose. The profile sits inside the user document because it is always loaded and saved along with the user, and this way the agent can get everything it needs in a single query. In the diagram each nested part of the profile is shown as one row with its fields listed beside it. The four degrees of the education history share the same set of fields. Applications, tracker entries and notifications can grow without limit, so they are kept in their own collections and refer to users and opportunities by ObjectId (marked FK). Every collection also records createdAt and updatedAt automatically.

Collection	Purpose	Key / index
users	One document per student: login data, the embedded profile (personal, address, education, education history, languages, statements, job, preferences, documents, saved answers), settings, and a password reset code (hash only) while a reset is in progress.	email (unique)
opportunities	Scholarships and jobs with details, status and cycle dates, eligibility and hard requirements, documents, how to apply, the recorded or learned form route, the questions the form asked, and the last form check.	source + url (unique)
applications	One document per agent run: status, step-by-step log, current pause (with any questions for the student), result and the form URL.	_id
tracks	Tracker entries: stage, notes, personal deadline and stage history.	user + opportunity (unique)
notifications	Bell messages with kind, text, link and read flag. key prevents duplicates (e.g. one deadline reminder per day).	user + createdAt

Passwords are never stored as plain text, only as bcrypt hashes. The same applies to a password reset code: only its hash is kept in users.reset, together with its expiry time, the number of wrong tries and when it was sent. Uploaded files are saved on disk in server/uploads/<userId>/, and the database only keeps each file's path and original name.

7. Entity Relationship Diagram (ERD)

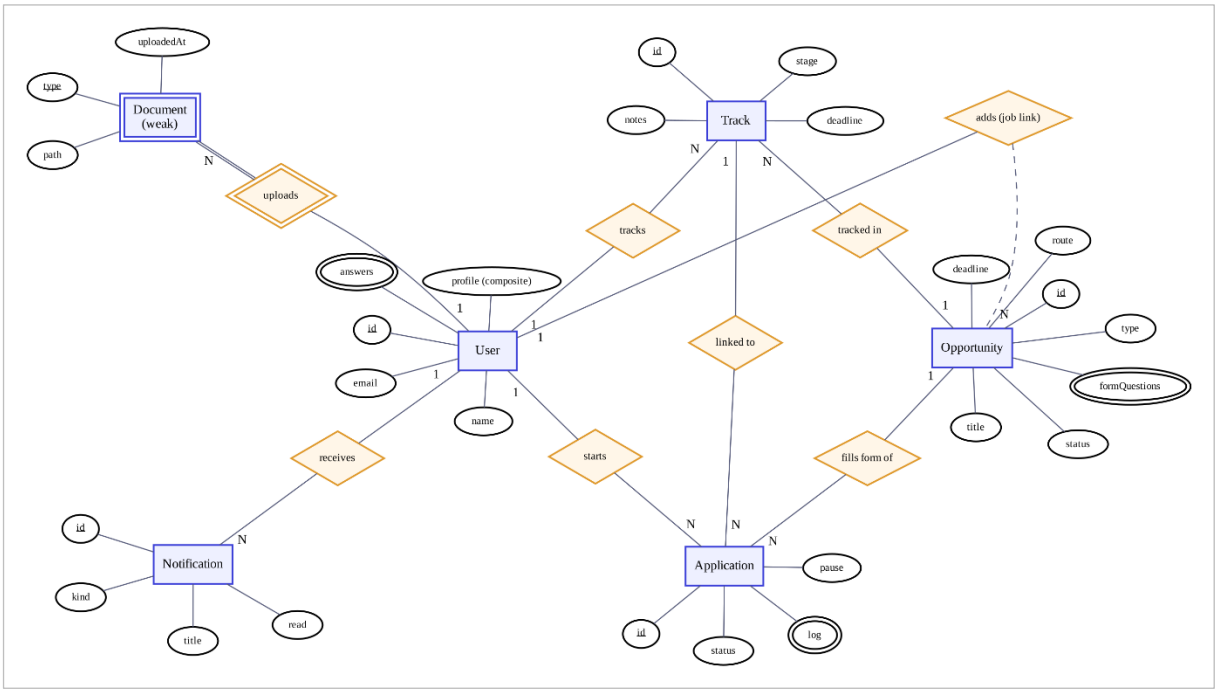


Figure 7.1: Entity Relationship Diagram (Chen notation)

The ERD uses Chen notation. Rectangles are entities, diamonds are relationships and ovals are attributes, with key attributes underlined. Document is drawn as a weak entity (double rectangle) because a document cannot exist without the user it belongs to. "log", "answers" and "formQuestions" are multivalued attributes and "profile" is a composite one. The table below lists each relationship and its cardinality.

Relationship	Cardinality	Description
User – uploads – Document	1 : N (max 8)	Document is a weak entity identified by its type within one user's profile.
User – starts – Application	1 : N	A student can run the agent many times; each run belongs to one student.
Application – fills form of – Opportunity	N : 1	Each run is for one opportunity; an opportunity can have runs from many students.
User – tracks – Track	1 : N	One tracker entry per opportunity the student saved.
Track – tracked in – Opportunity	N : 1	The pair (user, opportunity) is unique.
Application – linked to – Track	N : 1	Runs move the linked tracker entry forward.
User – receives – Notification	1 : N	Bell and email messages for one student.
User – adds – Opportunity	1 : N (optional)	Jobs added from a Rozee.pk link (a server feature, not shown on the Jobs page at present) are visible only to the student who added them.

8. Sequence Diagram

We have chosen the four interactions that matter most in ApplyMate and drawn a sequence diagram for each.

8.1. Sign Up and Log In

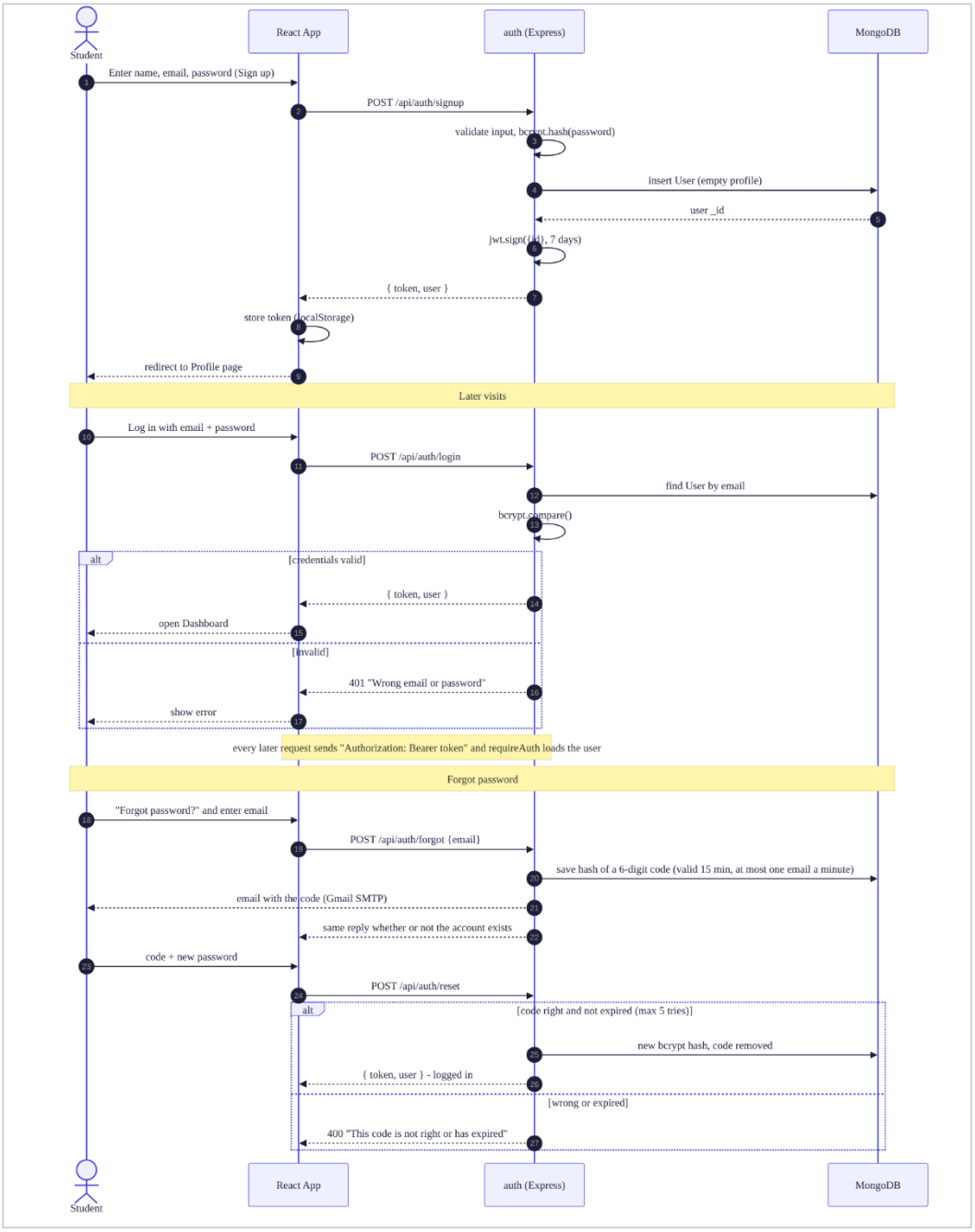


Figure 8.1: Sequence diagram for sign-up and login

When a student signs up, the password is hashed with bcrypt before it is saved. After sign-up or login the server sends back a token that stays valid for seven days. The browser keeps this token and attaches it to every request, and any request without a valid token is turned away. New users land on the Profile page so they can fill in their details first. A student who forgets the password asks for a 6-digit code by email. The code works for 15 minutes and for at most five tries, only one email is sent per minute, and the reply is the same whether or not an account exists for that email, so nobody can find out who is registered. With the right code the student chooses a new password and is logged in.

8.2. Apply with the AI Agent

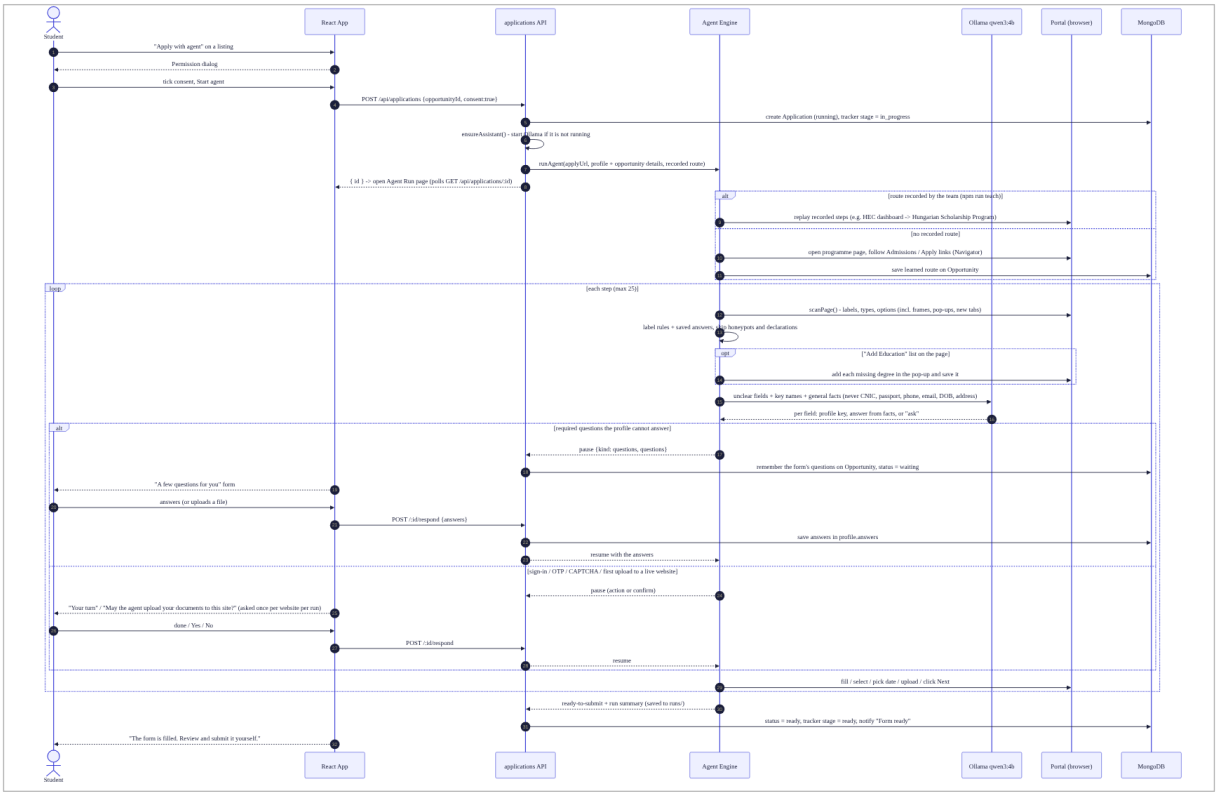


Figure 8.2: Sequence diagram for applying with the agent

This is the heart of ApplyMate. The server will not start a run until the student has ticked the consent box, and it first makes sure Ollama is running. The agent then gets to the form, by replaying a route recorded by our team, using a route saved by an earlier run, or searching from the programme's website. After that it keeps scanning, deciding and filling. Fields not settled by label rules or saved answers go to the local LLM with the profile key names and a few general facts, never personal values such as the CNIC or passport number. If the page has an "Add Education" list, the agent adds each missing degree. When the form asks required questions the profile cannot answer, the run changes to "waiting" and the questions are shown inside ApplyMate.

The answers are saved in the profile and given back to the agent. Before the first upload to a live website the agent asks once whether it may upload the student's documents there; the answer holds for the rest of that run. Sign-in, verification codes and CAPTCHAs are still handled by the student. A successful run ends at "ready to submit". The student then checks the form and clicks Submit themselves.

8.3. Import Profile from CV

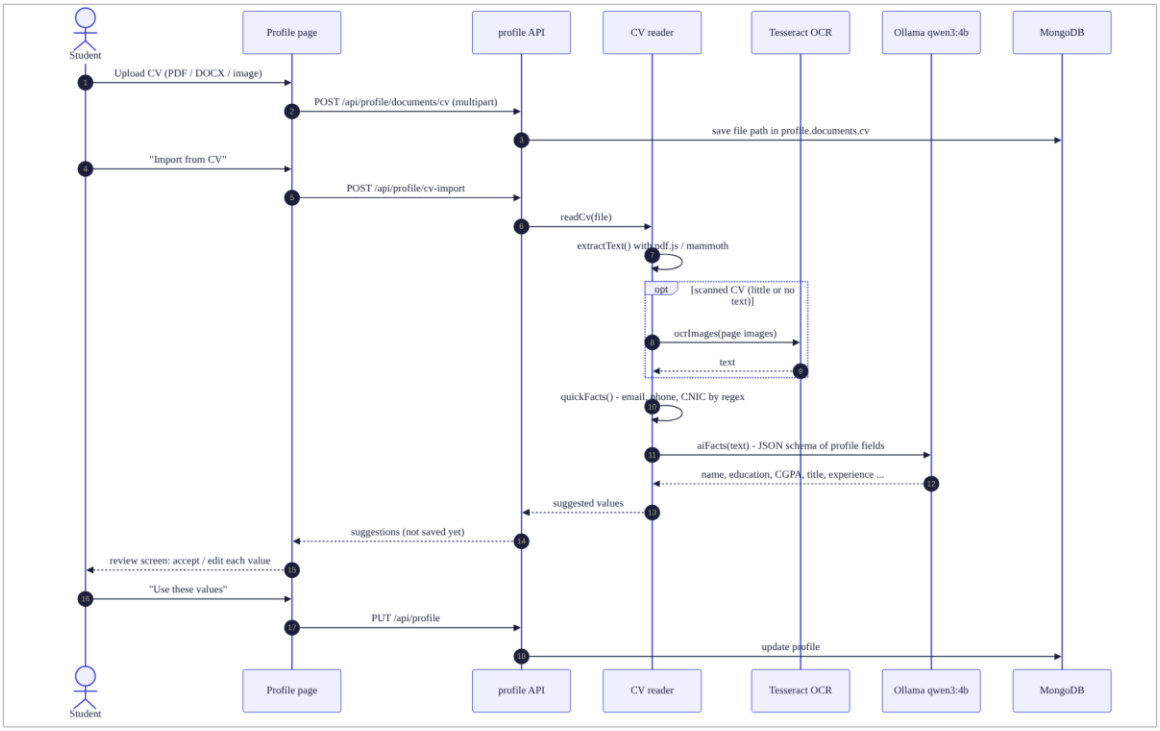


Figure 8.3: Sequence diagram for CV import

The text is taken out of the PDF or Word file, and if the CV is a scan, Tesseract OCR reads it instead. Email, phone and CNIC are picked up with simple patterns, and the LLM fills in the rest according to a fixed JSON schema. None of this is saved straight away. The student sees each suggested value next to the current one and decides what to keep.

8.4. Ask ApplyMate Chatbot

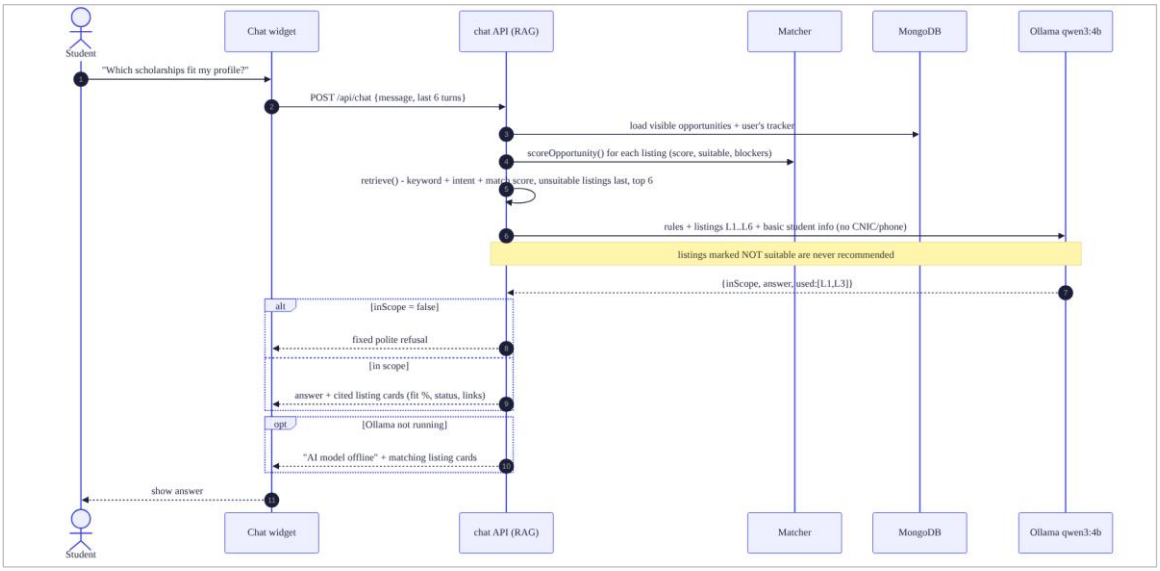


Figure 8.4: Sequence diagram for an Ask ApplyMate question

The chatbot works by retrieval-augmented generation (RAG). For each question the server first picks the six listings that fit best, based on keywords, the type of question and the student's match scores, with unsuitable listings placed last. The LLM is then told to answer using only those listings, never to recommend a listing marked as not suitable, and to say whether the question is about ApplyMate at all. Off-topic questions get a short, polite refusal. If Ollama happens to be off, the student still sees the matching listings.

9. System Interface Design

We kept the design simple and the same across every page. Navigation is in a sidebar on the left, content sits on white cards over a light background, and indigo (#3B3FD8) is the main colour. Saffron is used only when the student needs to do something, and green shows success. The screenshots in this section were taken from the working prototype using a sample student profile.

9.1. Landing Page

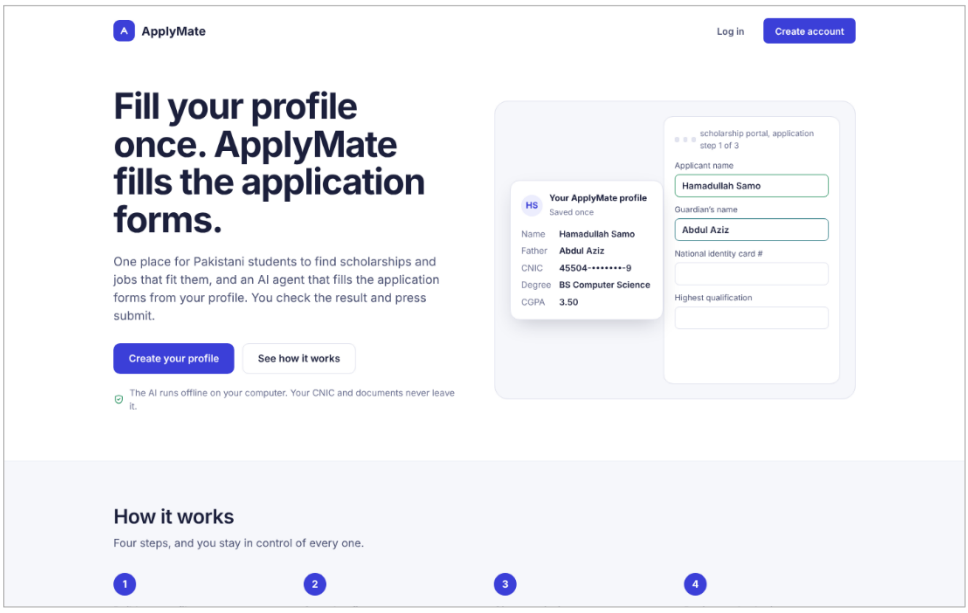


Figure 9.1: Landing page

This is the first page a visitor sees. It explains the idea in a single line, walks through the four steps of using ApplyMate, and links to sign-up and login. It also mentions that the AI works offline and the student's CNIC and documents stay on their computer.

9.2. Login and Sign-up

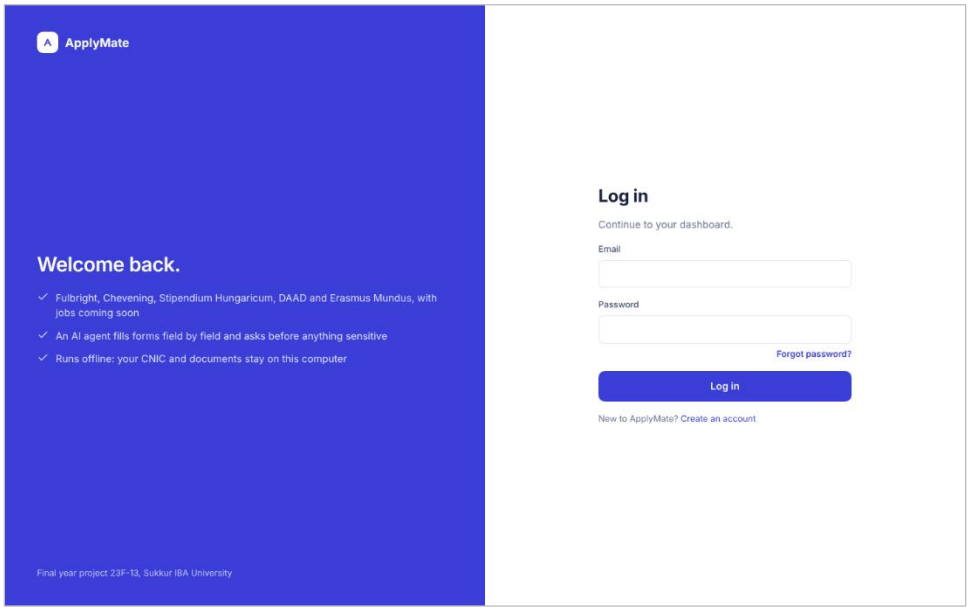


Figure 9.2: Login page (the sign-up page uses the same layout)

The page is split in two, with a short message about ApplyMate on the left and the form on the right. Sign-up only asks for a name, email and a password of at least six characters. Everything else is added later on the Profile page or imported from a CV. The login form has a "Forgot password?" link (see 9.16).

9.3. Dashboard

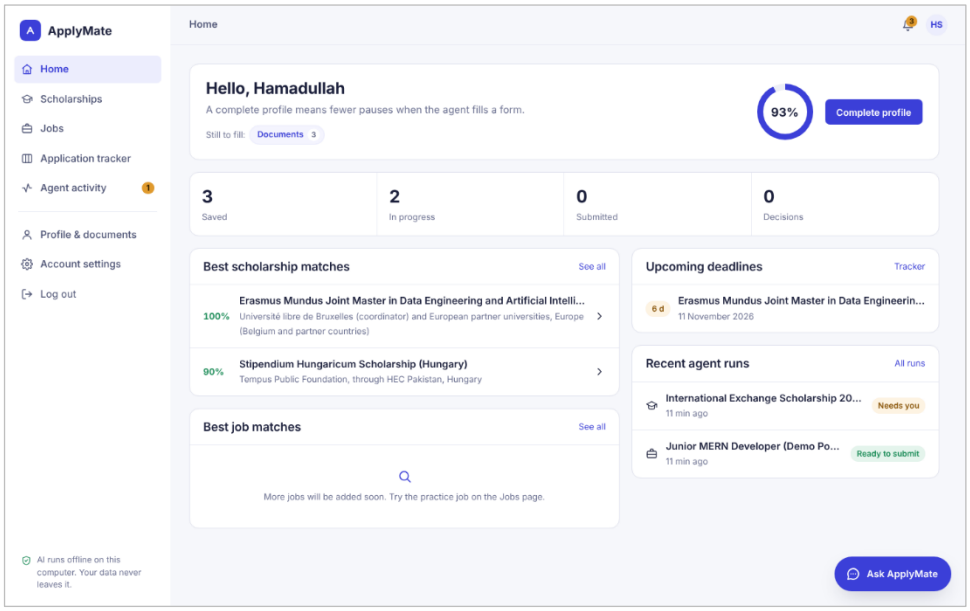


Figure 9.3: Dashboard (Home)

The dashboard greets the student by name and gives a quick overview: how complete the profile is, how many applications are at each stage, the best scholarship and job matches, upcoming deadlines and the latest agent runs. Profile completeness is counted section by section against what scholarship forms ask for, and a "Still to fill" line links straight to each section that has empty fields (here, the documents).

9.4. Profile and Documents

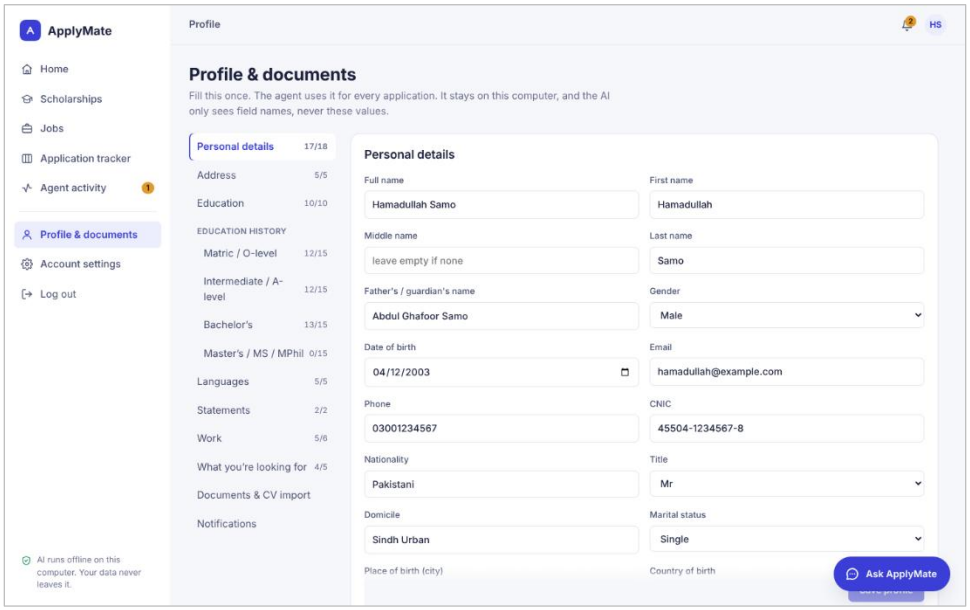


Figure 9.4: Profile and documents

The profile is divided into sections for personal details (including title, domicile, place of birth, marital status and passport), address, education, the education history (Matric, Intermediate, Bachelor's and Master's), languages, statements (motivation and career goals), work, preferences, documents and CV import, and notifications. Each section shows how many of its fields are filled. Whatever is entered here is reused in every application.

9.5. Scholarships

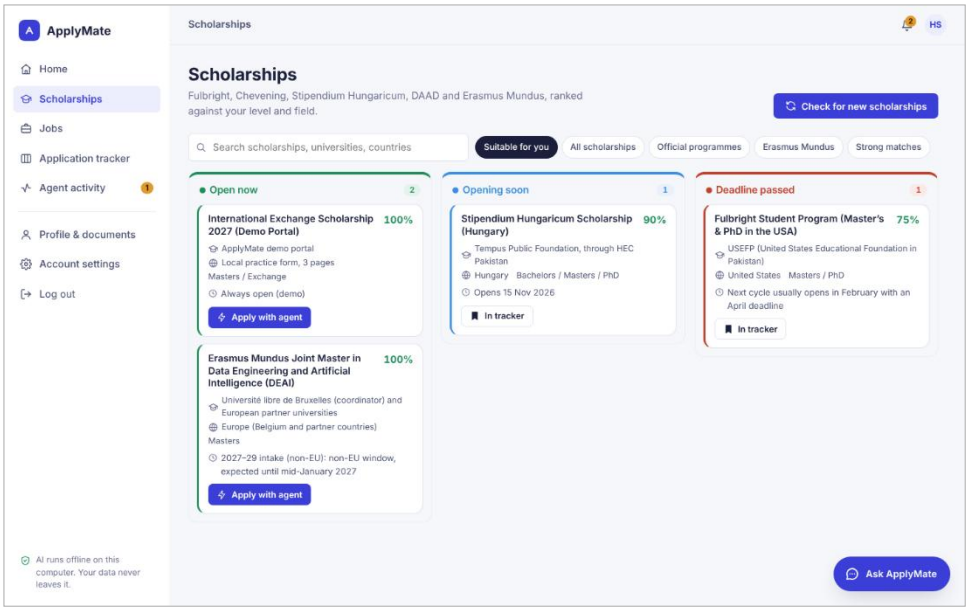


Figure 9.5: Scholarships board with match scores

Scholarships are arranged as a board with three columns: Open now, Opening soon and Deadline passed. Every card shows the match score, the programme and country, the study levels and its dates. Programmes that do not fit the student are marked as not suitable with the reason. Students can search, filter (for example "Suitable for you", "Official programmes", "Erasmus Mundus" or "Strong matches"), load new programmes, save a scholarship to the tracker or start the agent. A small badge shows whether our latest form check could reach the application form.

9.6. Jobs

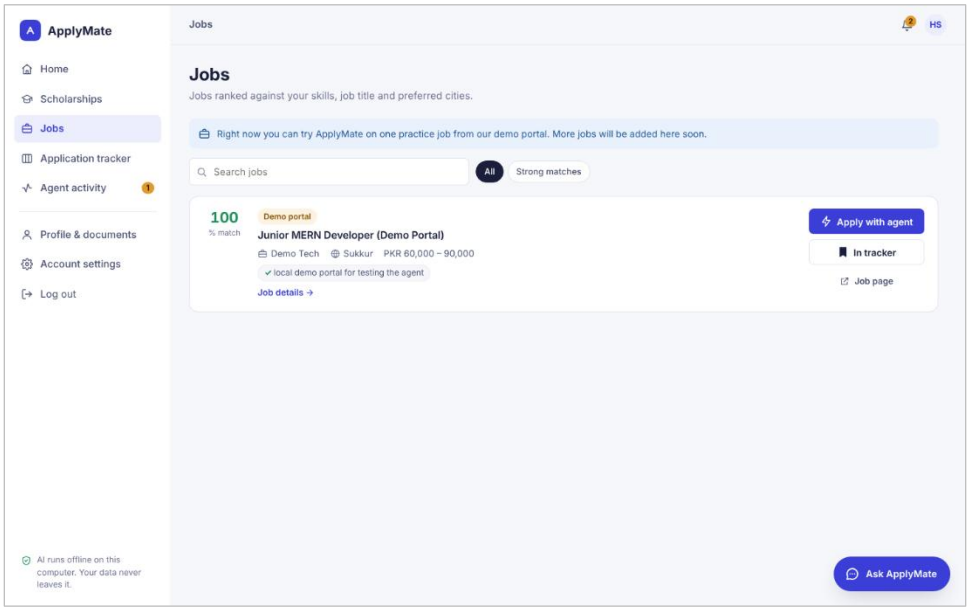


Figure 9.6: Jobs page with the practice job

The Jobs page ranks jobs against the student's skills, job title and preferred cities. In the current version it shows one practice job on our demo portal, so the agent can be tried on a job form, with a note that more jobs will be added soon.

9.7. Opportunity Details

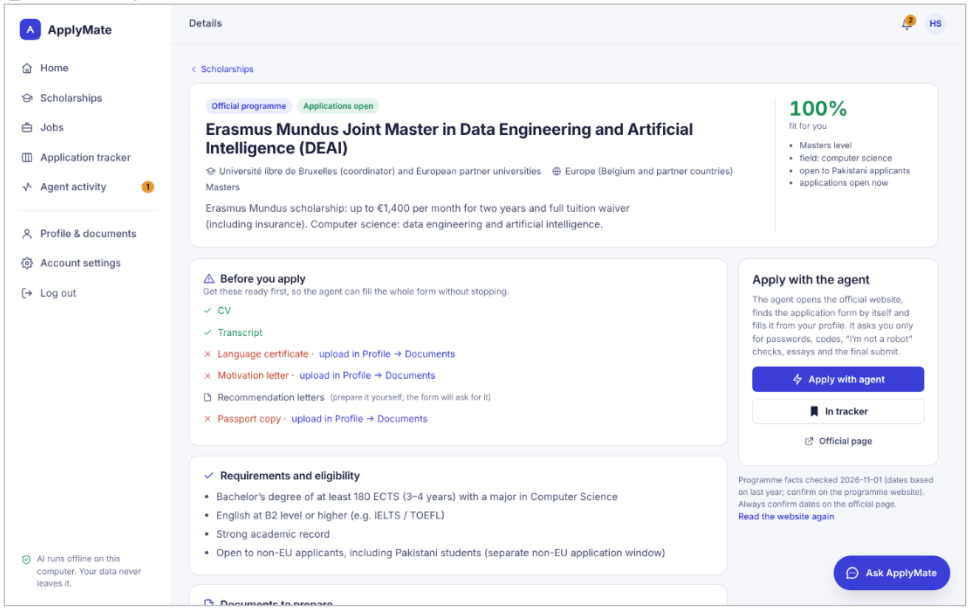


Figure 9.7: Opportunity details page

This page collects everything about one programme: a short summary, who is eligible, important dates and how to apply. A "Before you apply" box lists the documents the programme needs and shows which ones are already in the profile. Under "What this form will ask you" it lists the questions this programme's form has asked before, in short and readable words; the student can answer them here once, and the answers are saved in the profile for other forms too. The fit score and the "Apply with agent" button are on the right.

9.8. Permission Dialog

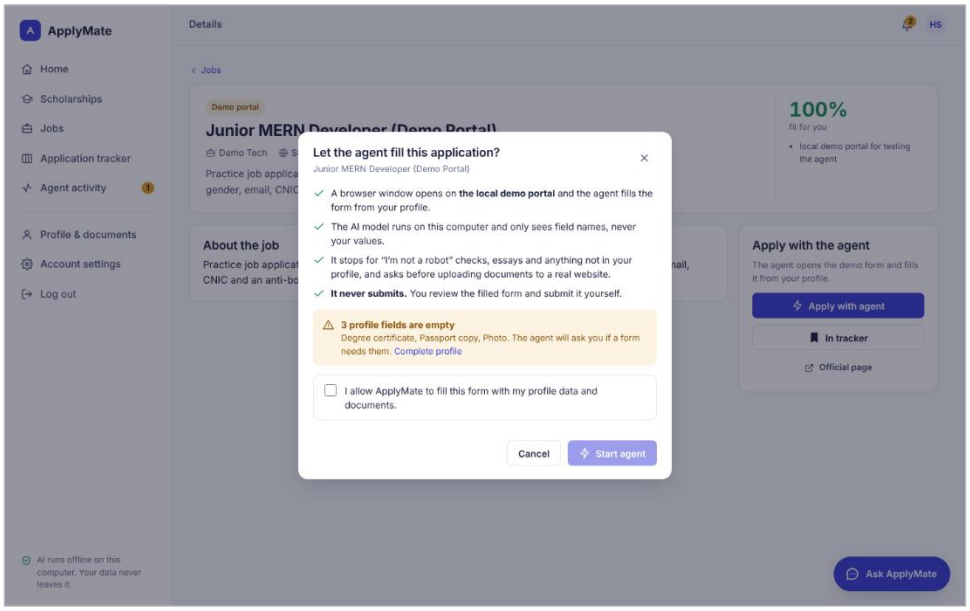


Figure 9.8: Permission dialog before the agent starts

Before each run the student is told what the agent will and will not do. If profile fields that forms usually ask for are still empty, the dialog says which ones, so the student knows the agent may stop to ask. The Start button only becomes active once they tick "I allow ApplyMate to fill this form with my profile data and documents".

9.9. Agent Run: Ready to Submit

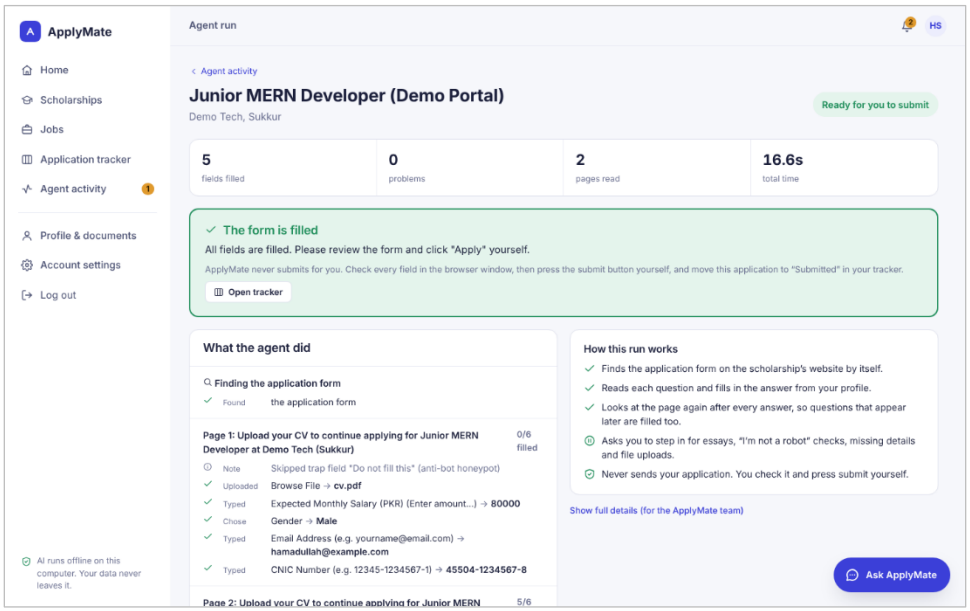


Figure 9.9: Agent run page after the form is filled

While the agent works, this page shows live progress: fields filled, problems found, pages read and time taken, along with a step-by-step list of what the agent did on each page. When it finishes, a green message asks the student to check the form and submit it themselves. If a run could not finish, the student sees a plain explanation and a "Try again" button.

9.10. Agent Run: Questions for You

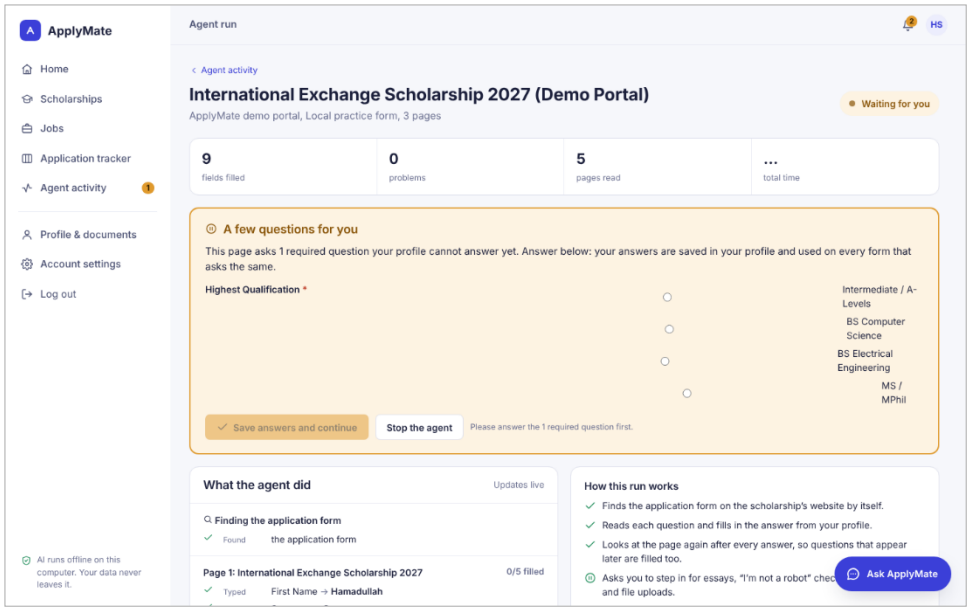


Figure 9.10: Agent asking the student's questions inside ApplyMate

When a form asks required questions that the profile cannot answer, the agent pauses and a "A few questions for you" box appears inside ApplyMate, so the student does not have to look for the field in the browser. The student answers (or uploads a file) and clicks "Save answers and continue". The answers are saved in the profile and used on every form that asks the same question. For things only the student can do in the browser, such as sign-in or a CAPTCHA, the "Your turn" box is still used.

9.11. Application Tracker

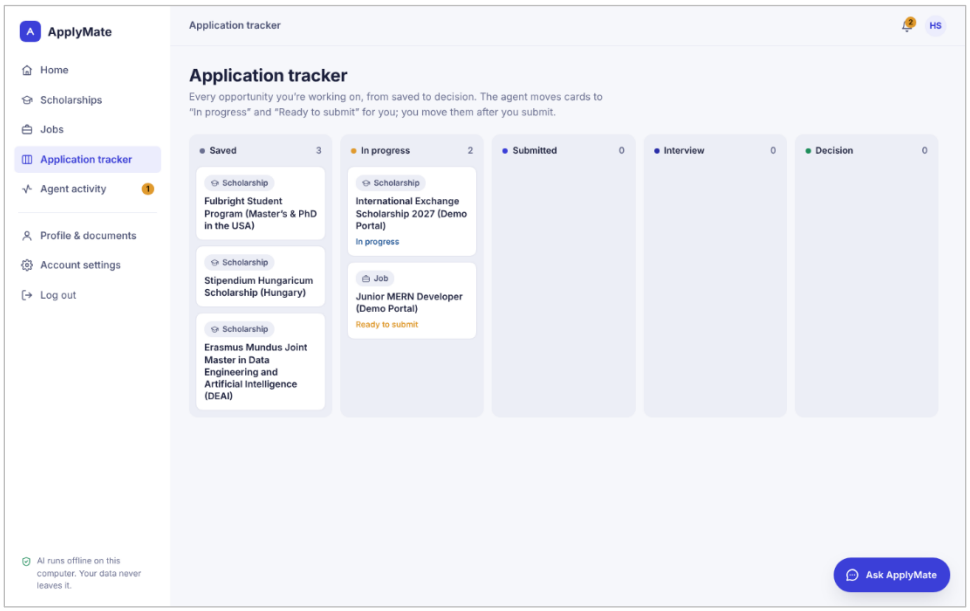


Figure 9.11: Application tracker (Kanban)

The tracker has columns for Saved, In progress, Submitted, Interview and Decision. The agent moves cards forward on its own while it works, and the student moves them after submitting. Each card can also hold notes and a personal deadline.

9.12. Agent Activity

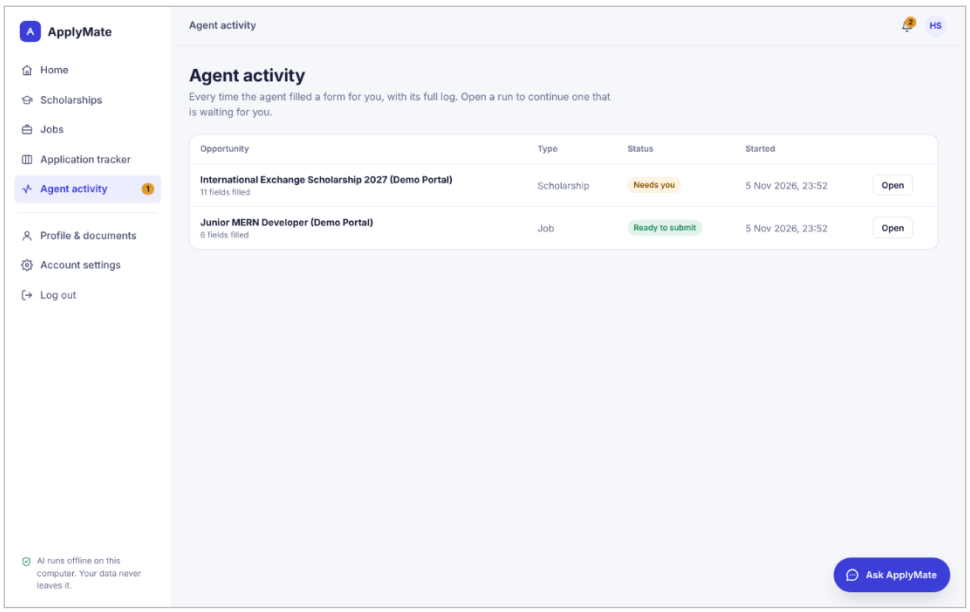


Figure 9.12: Agent activity history

A simple list of every agent run, showing the opportunity, its type, the result and when it started. Clicking a run opens its full log.

9.13. Notifications

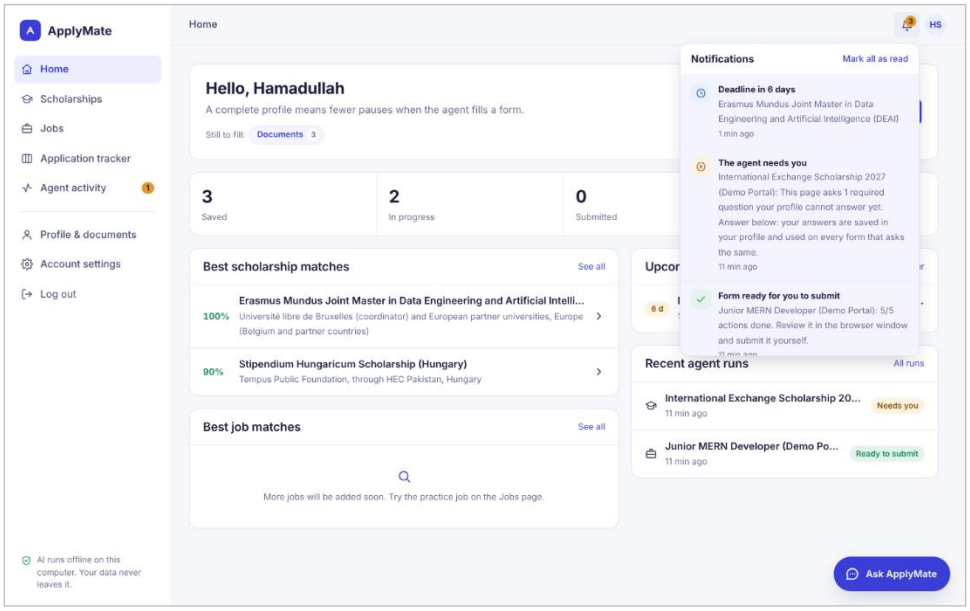


Figure 9.13: Notification panel

The bell collects reminders for deadlines in the next seven days, along with messages such as "The agent needs you" and "Form ready for you to submit". Students who turn on email notifications get the same messages in their inbox.

9.14. Ask ApplyMate Chatbot

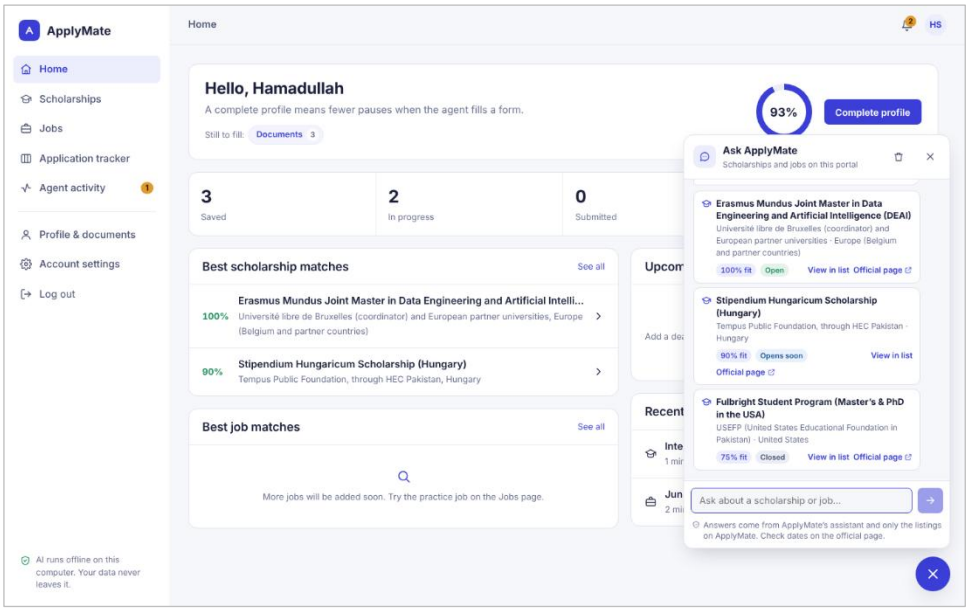


Figure 9.14: Ask ApplyMate chatbot

The chat button sits in the corner of every page. Each answer is followed by cards for the listings it was based on, with links to the listing and the official page. It never recommends a listing that is not suitable for the student. If a question has nothing to do with ApplyMate, the bot says so politely.

9.15. Education History in the Profile

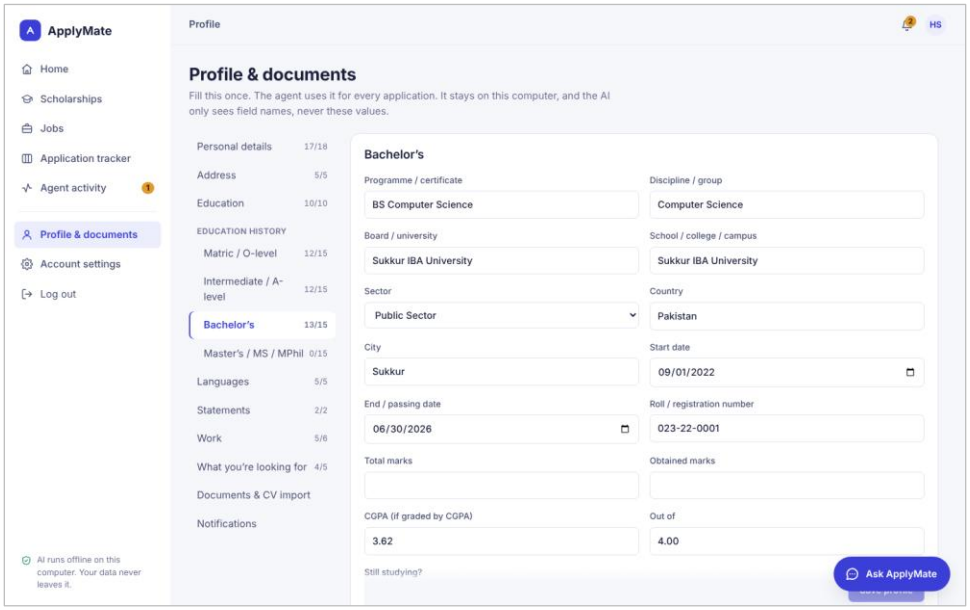


Figure 9.15: Education history (Bachelor's degree) in the profile

Many portals, the HEC Scholarship Portal in particular, ask for every degree separately. The profile therefore has an education history with one page each for Matric / O-level, Intermediate / A-level, Bachelor's and Master's. For each degree the student enters the programme, discipline, board or university, school, sector, city and country, dates, roll number, and marks or CGPA, and can mark a degree that is still in progress. The agent uses these entries to add the degrees one by one on the portal.

9.16. Forgot Password

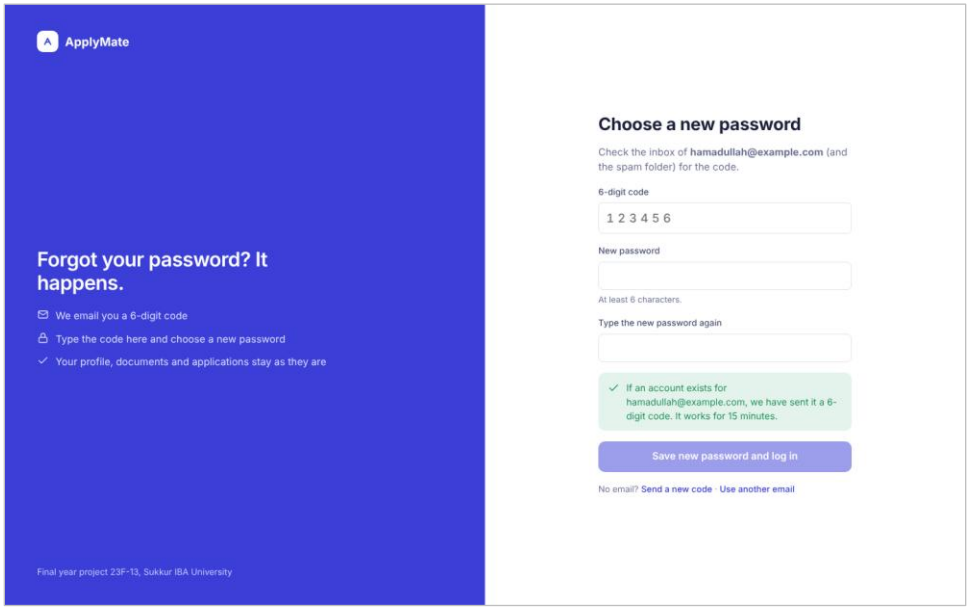


Figure 9.16: Forgot password, second step (enter the code and a new password)

A student who cannot log in clicks "Forgot password?" on the login page and enters their email. ApplyMate emails a 6-digit code that works for 15 minutes. On the second step, shown here, the student types the code and a new password twice, and is logged in straight away. The message is the same whether or not the email has an account, and the code can be requested again or sent to another email.

9.17. Account Settings

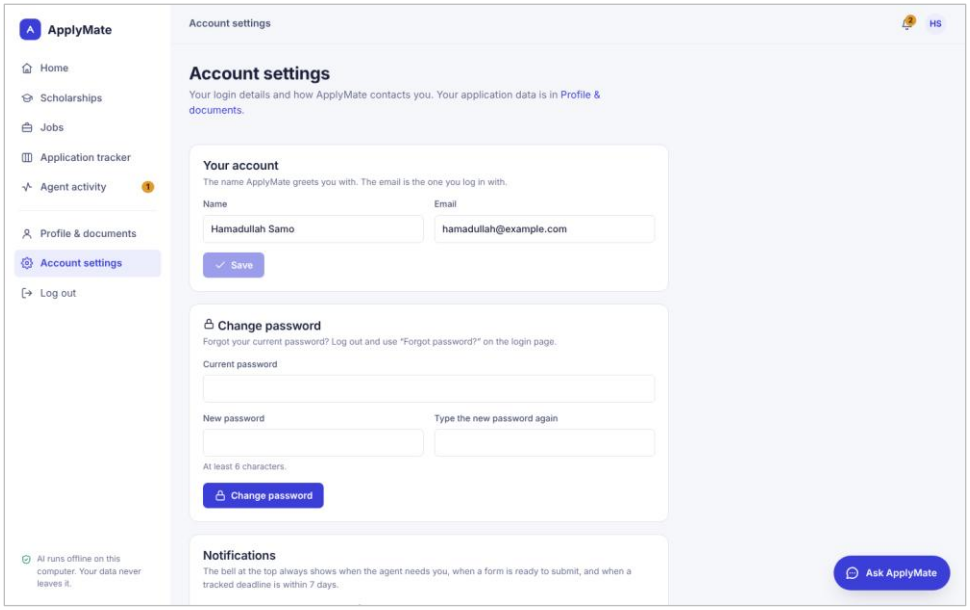


Figure 9.17: Account settings

Account settings is a separate page in the sidebar. The student can change the name ApplyMate uses, change the password (the current password is needed), turn email notifications on or off, and log out. The profile data used in applications stays on the Profile page.

10. Test Cases

The following test cases cover every module of the system. We will run them on our local demo forms and on the real portals, together with our supervisor. Each agent run also saves a JSON log in agent/runs/ with no personal values in it, and we will use these logs to measure how accurately fields are mapped and how much time is saved.

ID	Module	Test case / steps	Expected result
TC-01	Authentication	Sign up with a new name, email and a password of 6+ characters.	Account is created, a JWT is returned and the user is taken to the Profile page.
TC-02	Authentication	Sign up with an email that is already registered.	Error "An account with this email already exists." No new account is created.
TC-03	Authentication	Sign up with a 4-character password.	Error "Password must be at least 6 characters."
TC-04	Authentication	Log in with a correct email but wrong password.	HTTP 401 with "Wrong email or password." The user stays on the login page.
TC-05	Authentication	Call GET /api/profile without a token, or with an expired token.	Request is rejected with 401; the app returns to the login page.
TC-06	Profile	Fill personal, address, education, work and preference sections and click Save.	Values are saved and shown after reload; dashboard completeness increases.
TC-07	Documents	Upload a 2 MB PDF CV and a JPG photo.	Both files are saved and listed with their names and upload dates.

ID	Module	Test case / steps	Expected result
TC-08	Documents	Upload an .exe file, and a PDF larger than 5 MB.	Both are rejected: "Only PDF, Word, JPG or PNG files are allowed" / file too large.
TC-09	CV import	Upload sample-cv.pdf and click "Import from CV".	Suggested values (name, email, phone, degree, institution, CGPA, title) appear next to current values; nothing is saved until the user accepts.
TC-10	CV import	Import from a scanned (image-only) CV.	Text is read with OCR and suggestions are shown.
TC-11	Opportunities	Set preference level "Masters" and field "computer science"; open Scholarships.	Listings are sorted by match score; reasons such as "Masters level" are shown.
TC-12	Opportunities	Click "Check for new scholarships".	Erasmus Mundus programmes are loaded from the EACEA feed and added without duplicates.
TC-13	Opportunities	Open the Jobs page.	The practice job from the demo portal is listed with its match score, with a note that more jobs will be added soon.
TC-14	Opportunities	Call POST /api/opportunities/from-link with an invalid link, and with a Rozee.pk link that is not a single job page.	A clear validation error is returned for each and nothing is added.
TC-15	Agent	Click "Apply with agent" and try to start without ticking the consent box.	Start button is disabled; the API rejects a request without consent (400).
TC-16	Agent	Run the agent on the demo job form with a complete profile.	All fields and the CV upload are filled; status becomes "Ready for you to submit"; the form is NOT submitted.
TC-17	Agent	Run the agent on a form that contains a hidden honeypot field.	The honeypot is skipped and logged as "Skipped trap field".
TC-18	Agent	Run the agent on a form with a CAPTCHA.	Agent pauses with "Your turn"; after the student solves it and clicks continue, filling resumes.
TC-19	Agent	Run on a form with a Statement of Purpose / motivation field.	If a motivation statement is saved in the profile it is filled (and its length checked); otherwise the student is asked inside ApplyMate.
TC-20	Agent	Run on a live website form that asks for two documents on different pages.	The agent asks once "May the agent upload your documents to <site>?". On "Yes" both are uploaded without asking again; on "No" no uploads are made and the rest is filled.
TC-21	Agent	Run on a form with "I agree to the terms / I declare" checkboxes.	These boxes are left unticked for the student.
TC-22	Agent	Start a second run while one is running.	Rejected with "The agent is already working on another application" (409).
TC-23	Agent	Start a run while Ollama is installed but not running, and once while it cannot be started.	ApplyMate starts Ollama by itself and the run continues. If it cannot be started, the run stops with a plain message and a "Try again" button.
TC-24	Agent	Run a scholarship whose form route was saved by an earlier run.	The agent goes straight to the saved form; after 3 failed uses the route is searched again.
TC-25	Tracker	Save an opportunity, then on its tracker card change the stage from Saved to Submitted and add notes.	Stage, notes and a history entry are saved and shown after reload.
TC-26	Tracker	Start an agent run on a saved opportunity.	Card moves to "In progress", then to "Ready" when the form is filled.

ID	Module	Test case / steps	Expected result
TC-27	Notifications	Set a tracker deadline a few days from today (within 7 days) and open the Dashboard.	The deadline appears under "Upcoming deadlines" and one "Deadline in N days" notification is created (not repeated on the same day).
TC-28	Notifications	Turn on email notifications and click "Send a test email" with SMTP configured.	The branded test email arrives in the user's inbox.
TC-29	Chatbot	Ask "Which scholarships fit my profile?"	A short answer based only on portal listings, with cited listing cards and fit scores.
TC-30	Chatbot	Ask "What is the capital of France?"	Polite refusal: it only helps with scholarships and jobs on ApplyMate.
TC-31	Security	Inspect the prompts sent to the LLM during a run and a chat question.	Prompts contain field labels, key names and general facts only; never the CNIC, passport number, phone, email, date of birth, street address or document contents.
TC-32	Security	Request another user's application or tracker entry by its id.	Returns 404; users can only access their own records.
TC-33	Agent	Run on a form with a required question the profile cannot answer and the facts do not settle (e.g. "How did you hear about this programme?").	The run pauses and "A few questions for you" appears in ApplyMate; after "Save answers and continue" the answer is filled and the run goes on.
TC-34	Agent	Run a second form that asks the same question.	The saved answer is reused without asking again.
TC-35	Agent	On the HEC Scholarship Portal education page, run with Matric, Intermediate and Bachelor's saved in the education history.	Each missing degree is added through the "Add Education" pop-up and saved; degrees already listed are not added again; the final Submit is not clicked.
TC-36	Agent	Run a DAAD programme with a recorded route (npm run teach).	The agent replays the recorded steps to the application form after the student signs in, and never leaves the programme's portal.
TC-37	Agent	Fill a date field that uses a calendar picker.	The date is entered in a format the field accepts (the agent tries formats until one is accepted).
TC-38	Agent	Save a motivation statement longer than a form field allows and run the agent.	The text is not cut; the student is told the field's limit and asked to shorten it.
TC-39	Opportunities	Set work experience to 0 years and open a programme that needs 2+ years (or one only for former scholarship holders).	The programme is marked not suitable with the reason, its score is at most 30, and the chatbot does not recommend it.
TC-40	Opportunities	Run "npm run check-forms" and open Scholarships.	Each open scholarship shows "Form reachable", "Sign-in needed first" or "Form not found yet"; nothing is filled on any form.
TC-41	Opportunities	Open the detail page of a programme whose form has asked questions before.	"Before you apply" lists the documents (marking those already uploaded) and the questions, which can be answered and saved in advance.
TC-42	Profile	Add Matric, Intermediate and Bachelor's in the education history and save.	All degrees are saved with their dates and marks or CGPA and shown after reload.
TC-43	Authentication	On "Forgot password?", enter a registered email (email set up on the server).	A 6-digit code arrives by email; with the code and a new password the student is logged in, and the old password no longer works.
TC-44	Authentication	Enter a wrong code, an expired code, a used code, and a wrong code 5 times.	"This code is not right or has expired" each time; after 5 wrong tries even the right code is refused until a new one is requested.

ID	Module	Test case / steps	Expected result
TC-45	Authentication	Ask for a code for an email that has no account.	The same message is shown as for a real account, so nobody can tell which emails are registered.
TC-46	Authentication	Ask for a code on a server where email is not set up.	A clear message that password reset by email is not set up yet (503).
TC-47	Settings	Change the name in Account settings and save.	The new name is saved and used in the dashboard greeting and the initials at the top right.
TC-48	Settings	Change the password with a wrong current password, then with the right one.	First: "Your current password is not right." Second: the password is changed and the new one works at login.
TC-49	Opportunities	Set the field of study to "BSCS" and open Scholarships.	Computer science programmes match as if "computer science" had been written ("field: computer science").
TC-50	Dashboard	Leave some documents and the languages section empty and open the dashboard.	Completeness drops, and "Still to fill" lists each incomplete section with a link to it.
TC-51	Agent	Open the permission dialog with some profile fields empty.	The dialog lists the empty fields and warns that the agent may stop to ask for them.
TC-52	Opportunities	Run the agent on a form whose upload widget has an automatic name, then open the programme's detail page.	"What this form will ask you" shows only real questions, without broken labels or duplicates.